

操作系统内核技术文档

wllos

feat/ltp

华南师范大学

wll_OS 项目组

2026 年 6 月 26 日

目录

第 1 章 摘要	1
1.1 项目定位	1
1.2 设计边界	1
第 2 章 系统概述	3
2.1 总体架构	3
2.2 代码结构	4
2.3 启动与初始化流程	4
2.4 运行与评测路径	5
第 3 章 内存管理模块	6
3.1 设计目标	6
3.2 地址空间结构	6
3.3 映射与权限维护	7
3.4 brk、mmap 与共享内存	8
3.5 ELF 装载与用户栈	8
3.6 写时复制与缺页路径	9
第 4 章 进程与调度模块	11
4.1 任务模型	11
4.2 clone 与 execve 生命周期	11
4.3 等待队列与阻塞语义	12
4.4 退出、等待与父子关系	13
4.5 凭据与组管理	14
第 5 章 文件系统与 I/O 模块	15
5.1 VFS 与 fd 表	15
5.2 内存文件与大文件策略	15
5.3 权限、符号链接与路径解析	16
5.4 管道、poll 与非阻塞 I/O	16
5.5 ext4 与 MemFS 协作	17
5.6 向量 I/O 与固定偏移 I/O	17
第 6 章 系统调用与兼容模块	18
6.1 系统调用分发	18
6.2 用户内存访问	18
6.3 信号、futex 与时间接口	19
6.4 兼容性状态表	19
6.5 errno 策略	20
6.6 跨架构兼容细节	20
第 7 章 LTP 与评测模块	22
7.1 harness 设计	22

7.2 测试集合	23
7.3 有界 LTP 选择	23
7.4 parser 分组协议	23
7.5 新增 case 准入标准	24
第 8 章 测试结果与问题分析	25
8.1 编译与运行方式	25
8.2 当前能力总结	25
8.3 典型问题定位	26
8.4 回归证据记录格式	26
8.5 文档验收点	27
第 9 章 总结与展望	28
9.1 总结	28
9.2 后续工作	28

第 1 章 摘要

1.1 项目定位

wll0s 是一个面向操作系统内核竞赛与课程实践的 Rust 内核项目。`feat/ltp` 分支的主要目标不是一次性实现完整 Linux，而是在真实用户程序、真实 libc、真实文件系统镜像和真实评测日志的约束下，逐步扩大可运行的 Linux ABI 子集。本文档按照 UndefinedOS 文档的写法收束为摘要、系统概述、核心模块、测试结果和总结，不再把每个细碎主题拆成独立章节；模块内部则保留设计动机、数据结构、关键流程和代表性代码。

该分支的核心价值在于“可证明”。构建阶段通过 `Makefile` 控制 RISC-V64 与 LoongArch64 双架构，运行阶段通过 QEMU 串口日志暴露真实输出，评测阶段通过 `testdata` 中的 `judge` 脚本解析 `basic`、`busybox`、`lua`、`iozone`、`libc-test`、`libcbench`、`lmbench` 与 bounded LTP 的结果。内核不依赖伪造的通过标记，也不把未支持语义包装成完整能力，而是用一组边界清晰的系统调用与模块实现支撑当前测试面。

从工程形态看，wll0s 已经形成了较清晰的内核层次：启动代码负责架构入口和页表切换，内存模块负责地址空间和 VMA，任务模块负责任务生命周期和调度，文件系统模块负责 VFS、fd 与 ext4/MemFS 后端，系统调用模块负责 Linux ABI 入口，`harness` 负责在用户态测试集合和内核任务之间建立可重复的运行路径。本文重点描述这些模块之间如何协作，以及它们如何服务于 `feat/ltp` 分支的真实评测目标。

1.2 设计边界

当前分支更适合被理解为“竞赛评测内核”，而不是通用桌面或服务器内核。它支持真实 ELF 加载、动态解释器路径、用户栈构造、fork/clone/exec/wait 生命周期、VFS 路径操作、文件描述符、管道、部分 socket、部分 signal/futex/time 语义，以及有界 LTP case。它仍然保留明确边界：完整 TCP/IP、完整 pthread、完整 Linux signal queue、完整 ext4 journal、完整 page cache、完整权限能力模型和完整 LTP 都不是当前文档声称已经完成的内容。

本文中的“完整”一词只用于排版模板与文档结构，不表示内核已经拥有完整 Linux 能力。凡是当前实现属于 `partial` 或 `stub-risk` 的部分，文档会在对应模块中说明原因和后续方向。这样的写法虽然不如直接写“全部支持”漂亮，但对内核开发更有价值：后续同学能从文档直接知道哪些路径已经有真实对象支撑，哪些路径只是为了兼容当前测试而提供简化语义。

模块	功能
任务系统	维护任务、线程组、等待队列、信号状态和调度入口。
内存管理	提供页表、VMA、 <code>mmap</code> 、 <code>brk</code> 、SysV shm 和用户栈。
文件系统	组合 MemFS、ext4 运行时根盘、VFS、fd、pipe 和伪设备。
系统调用	按 Linux ABI 分发进程、文件、内存、信号、时间和网络相关请求。

模块	功能
评测 Harness	从运行时文件系统发现脚本，运行真实测试并保留串口证据。

第 2 章 系统概述

2.1 总体架构

wll0s 的总体结构可以分为五层。最底层是架构相关入口，包含 RISC-V64 与 LoongArch64 的启动汇编、链接脚本、SBI 或平台抽象相关初始化。其上为基础内核服务，包括堆分配器、物理页分配器、页表封装、定时器、日志和控制台。再往上是核心运行时，包括任务、地址空间、文件系统、trap 和系统调用。最上层是用户程序运行与评测 harness，它负责发现镜像中的脚本和二进制，按照 libc 根目录组织测试，并把串口输出整理成 judge 可识别的分组。

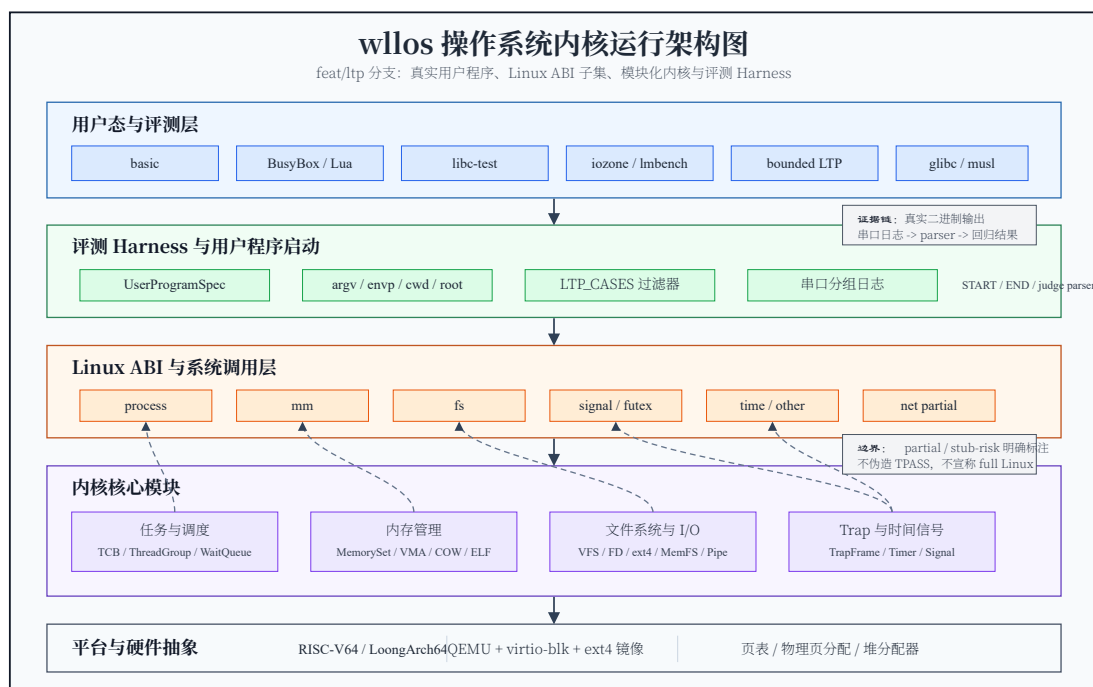


图 2-1 系统总体架构

这套结构的优点是职责边界较清楚。用户程序不直接依赖测试脚本的特殊路径，而是通过 `UserProgramSpec` 描述 `argv`、`envp`、`cwd`、`root` 和 `marker`；系统调用层不直接操作磁盘镜像，而是通过 `fd/VFS` 后端统一读写；任务调度不让每个子系统自己维护阻塞状态，而是用 `wait queue` 统一描述 `IO`、`child exit`、`timer`、`futex` 和 `signal` 的等待原因。模块之间仍有耦合，例如 `execve` 同时触碰 `ELF`、`MemorySet`、`fd close-on-exec`、`signal action` 和 `trap frame`，但这些耦合集中在少数生命周期函数里，便于定位。

2.2 代码结构

项目根目录中的 `ltp/` 是从 `feat/ltp` 分支拉取的源码。它同时保存了内核代码、构建脚本、评测脚本、文档和离线依赖。当前技术文档不复制整个文件树，只列出对理解内核最重要的目录。

路径	作用
<code>os/src/main.rs</code>	内核初始化入口，完成日志、内存、设备、文件系统、任务系统与 harness 启动。
<code>os/src/mm</code>	内存管理模块，包含页表、物理页分配、VMA、ELF 加载、COW 和映射维护。
<code>os/src/task</code>	任务管理模块，包含任务控制块、调度器、等待队列、pid、上下文和评测 harness。
<code>os/src/fs</code>	文件系统模块，包含 fd 表、VFS、MemFS、ext4 后端、pipe、socket 与伪设备。
<code>os/src/syscall</code>	系统调用模块，按照 Linux syscall id 分发到 process、fs、mm、signal、other、net 等子模块。
<code>os/src/trap</code>	异常和系统调用 trap 处理，保存当前 syscall 上下文，处理 <code>execve</code> 后的返回路径。
<code>scripts</code>	基线运行、评测汇总、镜像修复和测试套件补丁脚本。
<code>testdata</code>	judge 脚本与配置，用于从串口日志中提取测试组结果。

2.3 启动与初始化流程

内核启动流程可以理解为从“平台可运行”逐步过渡到“用户程序可运行”。早期入口负责建立栈和基本地址空间，随后初始化日志、堆、物理页、文件系统和任务系统。等到块设备和运行时根目录准备好之后，harness 才会启动用户程序。这个顺序不能随意调整：如果文件系统尚未挂载，动态解释器和测试脚本无法读取；如果任务系统尚未初始化，用户程序即使装载成功也无法进入调度；如果 timer 没有准备好，sleep、poll、futex timeout 和 interval timer 都会失去基础。

阶段	主要对象	说明
早期入口	链接脚本、启动栈、trap	完成架构相关最低限度初始化，使 Rust 内核代码可以运行。
内存初始化	heap、frame allocator、kernel page table	为后续动态对象、页表映射和用户地址空间分配提供基础。
设备与文件系统	virtio-blk、ext4、MemFS	建立运行时根目录，使用户程序、动态库和测试脚本可被读取。
任务系统	processor、task manager、wait queue	创建 harness 或 init 任务，并准备调度循环。
用户态运行	ELF loader、syscall、trap frame	进入真实测试程序，由系统调用和异常路径支撑后续执行。

```
pub fn rust_main() -> ! {
    logging::init();           // 串口日志是后续调试和 judge 的基础
    mm::init();                // 初始化堆和物理页分配器
    trap::init();              // 建立异常与系统调用入口
    drivers::init();           // 准备块设备等平台资源
    fs::init();                // 挂载 ext4, 准备 MemFS 和伪设备
    task::init_harness();      // 创建前台测试任务
    task::run_tasks();         // 进入调度循环, 开始运行用户程序
}
```

代码 2-1 内核启动主线示意

这段示意不是为了替代源码,而是把启动依赖关系压缩到一页中。`wllos` 的测试路径高度依赖串口输出,因此日志初始化要尽量早;用户程序依赖 `ext4` 镜像中的动态库,因此文件系统初始化必须早于 `harness`;任务调度依赖 `trap frame` 和地址空间激活,因此内存和 `trap` 初始化必须更早完成。这些依赖解释了为什么构建脚本、内核初始化和评测脚本需要一起看,而不能只看某个 `syscall` 是否存在。

2.4 运行与评测路径

`Makefile` 是 `feat/ltip` 分支的运行主线。它默认以 `RISC-V64` 为主架构,同时保留 `LoongArch64` 构建路径;通过 `LTP=1` 打开 `LTP feature`,通过 `LTP_CASES` 指定有界 `case` 集。与普通课程内核只运行一个 `init` 程序不同,这个分支的内核启动后会进入 `harness`,由 `harness` 按照镜像内容依次运行不同测试组。这样做的好处是所有测试都通过真实用户程序触发内核语义,缺点是内核需要承担更多 `Linux` 兼容细节。

```
LTP ?= 0
FOCUSED_LTP_CASES ?= writev01,setegid02,getgroups01,setgroups01,\
setgroups02,setgroups03,setgroups04,access01,open02,setfsuid01,\
setfsgid01,faccessat01,access02,open03,symlink01,readlink01,\
lstat01,lstat02,symlink02,symlink03,symlink04,symlinkat01,\
readlinkat01,setitimer02,setitimer01,getitimer01,getitimer02

ifeq ($(LTP),1)
    LTP_EXTRA := --features ltp
else
    LTP_EXTRA :=
endif
```

代码 2-2 构建脚本中的 LTP 控制入口

这段代码体现了当前 `LTP` 策略:默认不宣称完整 `LTP`,只在本地 `all` 或显式 `LTP=1` 路径中打开有界集合。`FOCUSED_LTP_CASES` 覆盖身份、组、访问权限、符号链接、`readlink/lstat` 和 `interval timer` 等有限主题,正好对应本分支近期补强的系统调用语义。后续扩展 `LTP` 时,应继续按子系统逐步增加,而不是直接运行完整脚本再用跳过或伪输出来掩盖失败。

第 3 章 内存管理模块

3.1 设计目标

内存管理模块需要同时满足三个目标。第一，能够装载真实 ELF，包括动态链接程序、解释器、用户栈、auxv 和 signal trampoline。第二，能够支持 libc 和测试程序常见的 `brk`、`mmap`、`munmap`、`mprotect` 与 SysV shared memory。第三，能够支撑 `clone`、`fork` 和 `execve` 的地址空间生命周期，尤其是 `fork` 后的写时复制和 `exec` 后的地址空间替换。

当前实现的主对象是 `MemorySet`。它持有一个根页表和一组 `MapArea`，每个 `area` 描述一段用户虚拟地址范围、权限和 `backing` 类型。匿名页、文件映射页、共享内存页都通过 `area` 进入统一维护路径。页表负责硬件可见映射，`area` 则是内核理解用户地址空间语义的来源。这个分离很关键：`mprotect`、`munmap` 和 `fork COW` 不应只改页表，还要同步修改 VMA 元数据，否则后续 `lazy fault`、用户拷贝或 `exec` 清理都会失去依据。

3.2 地址空间结构

```
/// 每个用户任务拥有或共享一个 MemorySet。
/// page_table 是硬件实际使用的根页表；
/// areas 是内核维护的 VMA 列表，用于 mmap、munmap、mprotect 和 fork。
pub struct MemorySet {
    pub page_table: PageTableWrapper,
    pub areas: Vec<MapArea>,
}

impl MemorySet {
    /// 创建一个空地址空间，通常用于 ELF 装载或 execve 替换。
    pub fn new_bare() -> Self {
        Self {
            page_table: PageTableWrapper::alloc_new(),
            areas: Vec::new(),
        }
    }
}
```

代码 3-1 `MemorySet` 的核心结构

在文档层面看，这段结构很小，但它贯穿整个系统。任务控制块中的 `memory_set` 使用 `Arc<Mutex<MemorySet>>` 包装，因此 `CLONE_VM` 可以让多个任务共享同一个地址空间；普通 `fork` 则通过 `fork_cow` 生成新的 `MemorySet`，并把可写私有页转换成 COW 权限；`execve` 会构造全新的 `MemorySet`，再替换当前任务的地址空间。这个选择使内核能用相对统一的对象模型覆盖进程、线程和测试 harness 的不同路径。

3.3 映射与权限维护

内存映射的实现重点不是“能 map 一页”，而是处理好边界。`insert_framed_area_with_backing` 会先对虚拟地址做页对齐，检查区间合法性，再根据权限决定是否立即分配物理页。对于 `PROT_NONE` 一类保留映射，`area` 可以先进入列表但不建立叶子页表项；等 `mprotect` 授予读写执行权限时，再补齐帧并重写页表。这样可以兼容 `libc` 的 TLS、DTV 扩展以及部分测试中先保留再启用的映射模式。

```
pub fn insert_framed_area_with_backing(
    &mut self,
    start_va: VirtAddr,
    end_va: VirtAddr,
    permission: PTEFlags,
    backing: MapAreaBacking,
) -> Result<(), SysErrNo> {
    let start = align_down(start_va.raw());
    let end = align_up(end_va.raw()).ok_or(SysErrNo::EINVAL)?;
    if start >= end {
        return Err(SysErrNo::EINVAL);
    }

    let mut area = MapArea::with_backing(
        VirtAddr::new(start),
        VirtAddr::new(end),
        permission,
        backing,
    );

    // 没有 R/W/X 权限时只记录 VMA，不立即建立叶子 PTE。
    if !has_leaf_permission(permission) {
        self.areas.push(area);
        self.coalesce_areas();
        return Ok(());
    }

    // 有实际访问权限时分配物理页，并把页表映射到用户地址。
    for vpn in start / PAGE_SIZE .. end / PAGE_SIZE {
        let frame = frame_allocator::alloc_frame().ok_or(SysErrNo::ENOMEM)?;
        let vaddr = VirtAddr::new(vpn * PAGE_SIZE);
        let paddr = PhysAddr::new(frame.ppn().addr());
        self.page_table.map_page(
            vaddr,
            paddr,
            effective_mapping_flags(permission),
            MappingSize::Page4KB,
        );
        area.frames.push(frame);
    }

    self.areas.push(area);
    self.coalesce_areas();
}
```

```
Ok(()  
}
```

代码 3-2 带 backing 的匿名或文件映射

这段摘录保留了实现的关键思想：VMA 和页表同时维护，失败时返回真实 `errno`，不把无效映射当作成功。这里的 `effective_mapping_flags` 还会处理 COW 页，将逻辑上可写但当前处于写时复制状态的页临时移除写权限，以便后续写异常触发复制。对 RISC-V 来说，`PROT_WRITE` 还需要配合读权限，因为硬件 PTE 不允许 W-without-R 的组合，这些架构差异被收敛在权限转换函数中。

3.4 brk、mmap 与共享内存

`brk` 主要服务 `libc` 分配器。当前实现维护逻辑 `break`、已映射 `break` 和 `heap` 起点：`brk(0)` 返回当前 `break`，合法增长会补充匿名页，合法收缩会释放上方页，不合法请求则保持旧 `break`。这种行为符合 Linux raw `brk` 的兼容预期，比简单返回错误更适合 `libc`。

`mmap` 覆盖匿名映射、部分文件映射和共享内存。匿名映射用于栈、堆外分配、动态链接器临时区域和 `pthread` 栈；文件映射用于动态库或测试程序读入文件内容；SysV shared memory 用于 `iozone` 和部分 `libc` 测试。当前文件映射不是完整 `page cache`：共享可写映射会在 `munmap` 或固定替换时写回，但缺少细粒度 `dirty tracking` 和完整 `msync`。文档中保留这个边界，是为了避免后续读者误以为它已经等价于 Linux。

接口	状态	说明
<code>brk</code>	<code>real</code>	维护真实堆区增长和收缩，非法请求保留旧 <code>break</code> 。
<code>mmap</code>	<code>partial</code>	匿名映射较完整，文件映射支持常见读写路径，但不是完整 <code>page cache</code> 。
<code>munmap</code>	<code>real/partial</code>	支持部分区间拆分和释放，共享文件映射写回策略较粗。
<code>mprotect</code>	<code>partial</code>	支持权限拆分与页表重映射，部分 Linux <code>personality</code> 语义未实现。
<code>shmget/shmat</code>	<code>partial</code>	提供真实共享物理页，覆盖 <code>iozone</code> 和基础 IPC 场景。

3.5 ELF 装载与用户栈

真实用户程序启动依赖 ELF loader。`wllos` 需要读取 ELF header 和 `program header`，把 `PT_LOAD` 段映射到用户地址空间，再处理动态链接程序的解释器路径。对于动态 ELF，内核并不是直接跳到目标程序入口，而是先装载解释器，把目标 ELF 的信息通过 `auxv` 传给解释器。`glibc` 和 `musl` 对 `auxv`、`argv`、`envp`、栈对齐和 `AT_PHDR/AT_PHNUM/AT_ENTRY` 都比较敏感，任何一个字段错误都可能表现为“程序还没开始就退出”。

```
pub fn new_user(elf_data: &[u8]) -> Result<Arc<Self>, SysErrNo> {  
    let elf = ElfFile::parse(elf_data)?;
```

```

let (mut memory_set, user_stack_top, entry) = elf.load()?;

// 在新地址空间的用户栈上写入 argc、argv、envp 和 auxv。
let sp = setup_user_stack_for_init(
    &memory_set,
    user_stack_top,
    entry,
    phdr_vaddr,
    phnum,
);

let mut trap_frame = TrapFrame::new();
prepare_user_trapframe(&mut trap_frame);
trap_frame[TrapFrameArgs::SP] = sp; // 用户栈顶
trap_frame[TrapFrameArgs::SEPC] = entry; // 用户入口地址
set_user_entry_registers(&mut trap_frame, sp, 1);

// signal trampoline 也要进入用户地址空间，否则信号返回无法恢复上下文。
signal::install_signal_trampoline(&mut memory_set)?;
/* 创建 TaskControlBlock, 注册到 task manager */
Ok(task)
}

```

代码 3-3 创建用户任务时的装载关键路径

这里的栈构造是启动链路中最容易被低估的部分。很多教学内核只需要把 `sp` 指向一段空栈，再把 `pc` 指向入口即可；但真实 `libc` 会从入口处解析 `argc/argv/envp/auxv`，动态链接器还会根据 `auxv` 定位目标程序头表、页大小、用户身份和随机种子。`feat/ltp` 分支为了运行 `glibc/musl` 测试，不得不把这些 ABI 细节纳入内存与进程模块的交界处。

3.6 写时复制与缺页路径

`fork` 直接复制所有用户页虽然简单，但在 `iozone`、`libcbench` 和多进程测试中会带来明显成本。`wll0s` 使用 COW 思路处理普通 `fork`：父子地址空间先共享只读物理页，并在页表权限中去掉写权限；当某一方写入时，通过缺页或写异常分配新页、复制内容并恢复写权限。这样既能保持 `fork` 语义，又能避免不必要的全量复制。

当前 COW 不是孤立功能，它和 `MapArea` 的 `backing` 类型、页权限、`frame ownership` 都有关。匿名私有页和私有文件映射适合 COW；共享内存和共享文件映射不应被错误复制；只读页可以直接共享；没有叶子权限的保留 VMA 则不能提前映射。文档把这些规则写在一起，是为了后续修改 `mmap` 或 `fork_cow` 时不破坏已有语义。

区域类型	fork 行为	原因
匿名私有页	COW	常见堆、栈和私有映射，写入时才复制。
只读文件页	共享	没有写权限，父子共享物理页即可。
共享内存	共享	语义要求多个进程观察同一物理内容。
共享文件映射	共享/写回	需要保持共享语义，当前写回粒度仍较粗。

区域类型	fork 行为	原因
<code>PROT_NONE</code> 保留区	仅复制 VMA	没有叶子页表项，授权时再分配或装载。

第 4 章 进程与调度模块

4.1 任务模型

wll0s 的任务模型围绕 `TaskControlBlock` 展开。一个任务既可以表示进程中的主线程，也可以表示 `CLONE_THREAD` 创建的线程。为兼容 Linux 语义，任务中同时保留 `pid`、`thread group`、父子关系、`fd` 表、`fs context`、`mm context`、`credentials`、`signal state`、`trap frame`、调度状态和等待状态。这个结构比教学内核中常见的“`pid + context + status`”复杂很多，原因是真实 `libc` 和 `LTP` 会频繁触碰这些边界。

```
pub struct TaskControlBlock {
    pub pid: Pid, // TID, 线程级身份
    pub thread_group: Arc<ThreadGroup>, // TGID, 进程级身份
    pub inner: Mutex<TaskControlBlockInner>,
    pub memory_set: SharedMemorySet, // CLONE_VM 时跨任务共享
    pub fs: SharedFsContext, // cwd/root/umask, 可被 CLONE_FS 共享
    pub mm: SharedMmContext, // brk 与 next_mmap 状态
    pub credentials: Mutex<Credentials>, // uid/gid/fuid/fsgid/groups
    pub signal_state: Mutex<SignalState>,
    pub trap_frame: Mutex<Option<TrapFrame>>,
    pub status: Mutex<TaskStatus>,
    pub block_reason: Mutex<Option<BlockReason>>,
    pub wait_outcome: Mutex<Option<WaitOutcome>>,
}
```

代码 4-1 任务控制块中的关键字段

这里最值得注意的是共享对象的粒度。Linux 的 `clone` 不是简单复制进程，而是通过 `flags` 控制资源共享。wll0s 用 `SharedMemorySet`、`SharedFdTable`、`SharedFsContext` 和 `SharedMmContext` 对应 `CLONE_VM`、`CLONE_FILES`、`CLONE_FS` 等语义；不共享时则深拷贝或复制上下文。这个模型让 `fork`、`vfork`、线程创建和 `harness` 前台任务都能走同一套生命周期函数。

4.2 clone 与 execve 生命周期

`sys_clone` 是进程模块中最重要的系统调用之一。它需要验证 `flag` 组合，复制当前 `trap frame`，让子任务返回值为 0，处理用户栈和 TLS，然后根据 `flags` 决定共享或复制地址空间、`fd` 表和 `fs context`。对于不共享地址空间的情况，当前实现使用 `fork_cow`，避免 `fork` 时立即复制所有物理页；对于 `CLONE_VM`，则直接共享父任务的 `MemorySet`。

```
let clone_bits = flags & !CSIGNAL;
if (clone_bits & CLONE_SIGHAND) != 0 && (clone_bits & CLONE_VM) == 0 {
    return Err(SysErrNo::EINVAL);
}
```

```

if (clone_bits & CLONE_THREAD) != 0 && (clone_bits & CLONE_SIGHAND) == 0 {
    return Err(SysErrNo::EINVAL);
}

let mut child_tf = crate::trap::clone_current_trapframe()
    .ok_or(SysErrNo::EINVAL)?;
child_tf[TrapFrameArgs::RET] = 0;          // 子任务从 clone 返回 0
child_tf.syscall_ok();                     // 子任务 PC 前进到 syscall 之后

let share_vm = (clone_bits & CLONE_VM) != 0 && !is_vfork;
let memory_set = if share_vm {
    parent.memory_set.clone()
} else {
    let mut parent_memory = parent.memory_set.lock();
    let child_memory = parent_memory.fork_cow()?;
    parent_memory.activate();
    new_shared_memory_set(child_memory)
};

```

代码 4-2 clone 中的资源共享选择

`execve` 则是另一条关键路径。它读取用户路径和参数，解析脚本或 ELF，处理解释器，装载新地址空间，重建用户栈和 `auxv`，关闭 `FD_CLOEXEC` 描述符，重置信号处理，并通知 `trap` 层不要再用普通 `syscall` 返回路径前进 PC。这里的细节直接影响 `BusyBox`、动态链接器、`libc-test` 和 `LTP` 的稳定性。尤其是 `/proc/self/exe`、解释器路径、环境变量和 `auxv`，如果处理不细，用户程序不是启动失败，就是在 `libc` 初始化早期崩溃。

4.3 等待队列与阻塞语义

真实测试中的阻塞不是只有 `sleep`。`pipe` 读写、`wait4`、`futex`、`timer`、`signal`、`poll/select` 都需要等待，并且要处理超时、信号中断、任务退出和过期唤醒。`wll0s` 通过 `WaitQueue` 统一这些场景，每个等待项带有 `task`、`token` 和可选 `key`。`token` 用于区分同一个任务不同时间进入的等待，避免一个旧 `timeout` 唤醒新等待。

```

pub fn sleep_until_key_if<F>(
    &self,
    key: Option<WaitKey>,
    deadline_us: Option<usize>,
    should_sleep: F,
) -> Result<WaitOutcome, SysErrNo>
where
    F: FnOnce() -> Result<bool, SysErrNo>,
{
    let task = current_task().ok_or(SysErrNo::ESRCH)?;
    let token = task.next_wait_token();
    *task.wait_outcome.lock() = None;
    *task.block_reason.lock() = Some(self.reason);

    self.waiters.lock().push_back(WaitEntry {

```



```

        task: task.clone(),
        token,
        key,
    });

    // 入队后再次检查条件，防止检查和睡眠之间丢失唤醒。
    if !should_sleep()? {
        self.remove_waiter(task.pid.0, token);
        *task.block_reason.lock() = None;
        return Ok(WaitOutcome::Woken);
    }

    if let Some(deadline) = deadline_us {
        crate::timer::add_timeout(deadline, task.clone(), token);
    }
    block_current_for(self.reason, deadline_us);

    *task.block_reason.lock() = None;
    let still_waiting = self.remove_waiter(task.pid.0, token);
    Ok(finish_wait(&task, still_waiting, deadline_us))
}

```

代码 4-3 等待队列的 token 化睡眠流程

这段代码体现了内核在阻塞语义上的工程取舍。它不追求完整 Linux 调度器，但尽量保证等待行为可解释：条件检查在入队后再执行一次，信号可中断等待，`deadline` 由 `timer` 子系统统一唤醒，任务退出时会清理相关等待项。对于 LTP 来说，这比“直接返回成功”更重要，因为许多 case 验证的正是 `errno`、超时和中断边界。

4.4 退出、等待与父子关系

进程退出路径需要同时处理线程级和进程级状态。普通 `exit` 只终止当前线程；当线程组中的最后一个用户线程退出时，整个进程才变为 `zombie`。`exit_group` 则要终止同一 TGID 下的所有用户线程，清理 `ready queue`、`futex queue`、`timer queue`、`IO wait queue`、`child wait queue` 和 `signal wait queue`，并向父进程投递 `SIGCHLD`。`wait4` 需要在父进程的子进程集合中查找 `zombie`，写回退出状态，并在没有可回收子进程但还有活子进程时阻塞。

这个设计比“子进程结束后直接从任务表删除”复杂，但它是 Linux 兼容所必需的。BusyBox shell、libc-test 和 LTP 都会假设父进程能通过 `wait` 系列调用观察子进程退出状态。如果内核提前释放任务，父进程会得到 `ECHILD`；如果不清理等待队列，已经退出的任务可能被旧 `timeout` 或旧 `wake` 再次唤醒；如果不处理 `SIGCHLD`，信号等待类测试会看不到子进程完成。

事件	状态变化	影响
<code>exit</code>	当前 TID 退出	清理线程私有状态，最后线程退出时标记进程 <code>zombie</code> 。
<code>exit_group</code>	整个 TGID 退出	终止同组用户线程，统一唤醒父进程和等待者。

事件	状态变化	影响
<code>wait4</code>	回收 zombie 子进程	写回 status, 释放父子关系, 避免 zombie 泄漏。
任务被 kill	从队列中移除	避免 ready/wait/futex/timer 中残留过期任务。
父进程退出	子进程交给 reaper	防止孤儿进程丢失回收路径。

4.5 凭据与组管理

bounded LTP 中的 `setegid02`、`getgroups01` 和 `setgroups01` 到 `setgroups04` 直接验证任务凭据。wll0s 因此为任务保存 real/effective/saved UID/GID、filesystem UID/GID 和 supplementary groups。`clone` 会继承凭据，`execve` 保留当前凭据，`setuid/setgid/setresuid/setresgid` 根据 root-capable 或已有身份集合决定是否允许切换。文件系统权限检查则可以选择 real identity 或 filesystem identity，这正是 `access` 与 `open` 的区别之一。

```
pub struct Credentials {
    pub real_uid: u32,
    pub effective_uid: u32,
    pub saved_uid: u32,
    pub fsuid: u32,
    pub real_gid: u32,
    pub effective_gid: u32,
    pub saved_gid: u32,
    pub fsgid: u32,
    supplementary_groups: Vec<u32>,
}

impl Credentials {
    pub fn is_in_filesystem_group(&self, gid: u32) -> bool {
        self.fsgid == gid
        || self.supplementary_groups.iter().any(|group| *group == gid)
    }
}
```

代码 4-4 任务凭据结构

当前凭据实现仍是能力模型的子集，没有完整 Linux capability 和 namespace 语义。但它已经足够让文件访问、组列表和有界身份切换不再是简单 stub。这个模块后续的扩展方向，应优先围绕更多权限矩阵和错误码，而不是马上引入复杂命名空间。

第 5 章 文件系统与 I/O 模块

5.1 VFS 与 fd 表

文件系统模块是 `feat/ltp` 分支中扩展最快的部分。它需要同时服务 BusyBox shell、动态链接器、libc 文件访问、iozone 大量读写、LTP 权限与符号链接 case。当前实现把文件对象抽象成 fd 表中的 `FileDescriptor`，再由不同后端提供读写、seek、stat、truncate、目录项、pipe、socket 和伪设备能力。这样的结构让 `openat`、`read`、`write`、`readv`、`writew`、`pread64`、`pwrite64`、`lseek`、`getdents64` 等接口都能通过同一套 fd 路径分派。

对象	代表路径	设计说明
MemFS	<code>/tmp</code> 、 <code>/var/tmp</code> 、 <code>overlay</code>	提供可写内存文件和白化覆盖，适合临时文件与测试脚本。
ext4	<code>/glibc</code> 、 <code>/musl</code> 根目录	承载真实用户程序、动态库、脚本和测试数据。
伪设备	<code>/dev/null</code> 、 <code>/dev/zero</code>	提供字符设备元数据和真实 <code>null/zero</code> 行为。
Pipe	<code>pipe2</code> 创建	有界内核缓冲区，支持阻塞与 <code>O_NONBLOCK</code> 。
Socket	<code>socket</code> 创建	提供本地 <code>loopback</code> 风格队列，外部网络仍非完整实现。

fd 表不仅保存文件对象，还保存 `offset`、`flags`、`close-on-exec` 和访问权限。`FD_CLOEXEC` 对动态链接器、shell 和测试框架很重要，因为 `execve` 后泄漏 fd 会改变程序行为。`O_APPEND`、`O_NONBLOCK`、`F_GETFD`/`F_SETFD`、`F_SETFL` 也会影响 BusyBox 和 LTP case。当前实现已经支持常见路径，但完整的共享 `open-file description` 仍然是后续方向。

5.2 内存文件与大文件策略

MemFS 的一个细节是大文件内容不总是使用单个 `Vec<u8>`。当文件超过阈值后，内容会转换为 `chunked` 表示，空洞 `chunk` 可以用 `None` 表示。这种设计在测试中很有用：iozone、libcbench 或临时文件可能产生较大缓冲，如果每次都复制完整连续数组，内核内存压力会很快上升。`chunked` 表示牺牲了一点实现复杂度，但换来了更可控的内存增长。

```
pub enum MemFileContent {
    Inline(Vec<u8>),
    Chunked {
        len: usize,
        chunks: Vec<Option<Vec<u8>>>,
    },
}
```

```
impl MemFileContent {
    pub fn from_slice(content: &[u8]) -> Self {
        if content.len() <= MEM_FILE_INLINE_LIMIT {
            return Self::Inline(content.to_vec());
        }

        let mut chunks = Vec::new();
        for part in content.chunks(MEM_FILE_CHUNK_SIZE) {
            if part.iter().all(|byte| *byte == 0) {
                chunks.push(None); // 全零块不实际保存
            } else {
                chunks.push(Some(part.to_vec()));
            }
        }
        Self::Chunked { len: content.len(), chunks }
    }
}
```

代码 5-1 MemFileContent 的分层存储

这种实现并不是完整的稀疏文件系统，但已经能减少测试场景中的重复零页开销。对文件系统模块来说，最重要的是读写路径要保持一致：普通 read/write、固定偏移 pread/pwrite、向量读写 readv/writev、stat/fstat/truncate 都应该看到同一份内容和长度。文档中把它放在文件系统章节，是因为它直接解释了为什么 MemFS 不只是一个简单 map，而是承担了测试期间的临时文件后端。

5.3 权限、符号链接与路径解析

bounded LTP 当前覆盖了 access01、faccessat01、open02/open03、symlink*、readlink*、lstat* 等 case。这些测试不只关心“文件是否存在”，还会验证父目录搜索权限、fsuid/fsgid、O_NOFOLLOW、空路径、符号链接循环、错误码和元数据。wll0s 因此在 VFS 层引入了更接近 Linux 的路径解析和权限检查，而不是在系统调用入口用字符串特判。

权限模型当前仍是简化 Linux 能力模型：任务带有 real/effective/saved uid/gid、fsuid/fsgid 和 supplementary groups，文件访问可以按 real 或 filesystem identity 判断；但完整 capability、ACL、mount namespace、sticky bit 等没有完整实现。这个边界对 LTP 扩展尤其重要。如果下一步加入更复杂的权限 case，应先补 VFS 权限模型，而不是继续在单个 syscall 中堆兼容判断。

5.4 管道、poll 与非阻塞 I/O

管道和 poll/select 是 BusyBox、shell、lmbench 与部分 libc 测试的基础。当前 pipe 使用有界内核缓冲区，读端无数据时可阻塞或在 O_NONBLOCK 下返回 EAGAIN，写端无读者时返回 EPIPE 并触发 SIGPIPE。ppoll 和 pselect6 通过 IO wait queue 睡眠，避免忙等。虽然还没有完整文件描述符共享状态模型，但 pipe 的可读可写、HUP、ERR 与非阻塞语义已经足够支撑当前测试面。

5.5 ext4 与 MemFS 协作

运行时镜像中的真实程序主要位于 ext4 根目录，例如 `/glibc` 和 `/musl`。这些目录承载 BusyBox、动态库、libc-test、LTP 二进制和各类脚本。MemFS 则更适合承担运行时生成内容，例如 `/tmp`、`/var/tmp`、测试临时文件、覆盖层和伪设备。两个后端通过 VFS 暴露统一路径接口，但持久化能力和元数据能力不同：ext4 更接近真实磁盘文件系统，MemFS 更像内核运行期的可写覆盖。

这种组合使内核既能读取评测镜像中的真实二进制，又能在测试期间创建、删除和修改临时文件。它也带来一个边界：跨后端 rename、link、symlink 和 metadata 同步需要谨慎处理，不能简单假设所有路径都属于同一种文件系统。当前实现已经覆盖常见同后端和 overlay 场景，完整跨挂载语义仍属于后续工作。

后端	优势	当前边界
ext4	真实镜像、真实目录和文件元数据	journal、完整 dirty tracking、复杂跨目录替换仍不完整。
MemFS	可写、快速、适合临时对象	元数据字段较少，权限和 owner 持久化有限。
伪设备	行为简单、兼容 libc 检查	只覆盖 <code>/dev/null</code> 、 <code>/dev/zero</code> 等有限设备。
Pipe/Socket	支撑 IPC 和 poll/select	外部网络和完整 socket blocking 仍缺失。

5.6 向量 I/O 与固定偏移 I/O

`writew01` 是当前 bounded LTP 的入口 case 之一，它验证的不只是 `writew` 是否存在，还包括用户 `iovec` 读取、分段长度求和、错误指针、部分写入和 `fd` 后端行为。`iozone` 又会进一步放大 `pread`、`pwrite`、`preadv`、`pwritev` 的性能问题。wllos 因此在 `syscall/fs` 路径中保留 `IoVec` ABI 结构，并把向量 I/O 合并到普通 `fd` 读写路径上。

```
#[repr(C)]
#[derive(Clone, Copy)]
struct IoVec {
    iov_base: *mut u8,
    iov_len: usize,
}

const SMALL_IO_STACK_BUF: usize = 4096;
const VECTORED_STACK_BUF: usize = 16 * 1024;
const VECTORED_IO_CHUNK: usize = 64 * 1024;
```

代码 5-2 用户态 `iovec` ABI

这几个常量反映了一个现实取舍：向量 I/O 不能为每个用户缓冲区都分配巨大临时内存，也不能在内核栈上放任大数组。实现中通常会把小 I/O 放在栈内缓冲，把较大的向量读写分块处理，再交给后端 `fd` 的 `read/write` 或 `read_at/write_at`。这样既能保持 ABI 兼容，又能避免测试压力下内核内存快速膨胀。

第 6 章 系统调用与兼容模块

6.1 系统调用分发

系统调用模块按 Linux syscall id 分发。`os/src/syscall/mod.rs` 保存了大量 syscall 常量，并把具体实现拆到 `fs.rs`、`mm.rs`、`process.rs`、`signal.rs`、`other.rs` 和 `net.rs`。这种拆分的好处是可以让每个子模块围绕自己的核心对象演进：进程调用操作任务和地址空间，文件调用操作 fd/VFS，内存调用操作 `MemorySet`，signal 和 `futex` 则操作任务等待状态。

```
pub type SyscallRet = Result<usize, SysErrNo>;

pub fn syscall(syscall_id: usize, args: [usize; 6]) -> SyscallRet {
    log::debug!("[syscall] id: {}, args: {:?}", syscall_id, args);

    match syscall_id {
        SYSCALL_OPENAT => fs::sys_openat(
            args[0] as isize,
            args[1] as *const u8,
            args[2],
            args[3],
        ),
        SYSCALL_CLONE => process::sys_clone(
            args[0], args[1], args[2], args[3], args[4],
        ),
        SYSCALL_MMAP => mm::sys_mmap(
            args[0], args[1], args[2], args[3], args[4], args[5],
        ),
        _ => Err(SysErrNo::ENOSYS),
    }
}
```

代码 6-1 系统调用分发入口示意

真实代码中的 `match` 分支比这里长得多。本文只保留代表性片段，是为了说明分发思路。系统调用返回值统一使用 `Result<usize, SysErrNo>`，这样成功路径直接返回寄存器值，失败路径由 `trap` 层转成 Linux 负 `errno`。对于不支持的 syscall，返回 `ENOSYS` 比伪造成功更可靠，因为测试和后续调试能清楚看到能力边界。

6.2 用户内存访问

系统调用经常需要读取用户指针，例如路径字符串、`argv/envp/iovec/stat` 结构、`timespec/sigset`。wll0s 把用户内存复制放在 `syscall/user` 辅助路径中，并在必要时切回内核页表、再恢复当前用户页表。这样做可以避免内核直接解引用用户地址导致页表上下文混乱，也便于对 `EFAULT` 做统一处理。

用户内存访问对 LTP 很敏感。许多 LTP case 会故意传入坏指针、零长度、边界地址或不可写缓冲区，期望内核返回精确 `errno`。如果内核把这些情况当成空字符串或静默忽略，就可能让普通程序看起来能跑，但 LTP 会在错误码上失败。当前文档把这部分放入系统调用章节，是因为它横跨 `fs`、`mm`、`signal`、`time` 等多个子模块。

6.3 信号、futex 与时间接口

`signal.rs`、`other.rs` 和 `timer.rs` 支撑了当前测试中较难的兼容点。信号模块实现了 `rt_sigaction`、`rt_sigprocmask`、`rt_sigsuspend`、`rt_sigtimedwait`、`rt_sigreturn`、`kill/tkill/tgkill` 的部分语义；futex 路径支持 `WAIT`、`WAKE`、`WAIT_BITSET`、`WAKE_BITSET` 和 robust list 的基础清理；时间路径支持 `nanosleep`、`clock_gettime`、`gettimeofday`、`times`、`getrusage`、`setitimer/getitimer` 等。

```
pub struct TaskControlBlockInner {
    pub interval_timers: IntervalTimers,
    pub default_timer_slack_ns: usize,
    pub current_timer_slack_ns: usize,
}

// LTP 的 setitimer/getitimer case 依赖三类 timer:
// ITIMER_REAL    到期后发送 SIGALRM;
// ITIMER_VIRTUAL 到期后发送 SIGVTALRM;
// ITIMER_PROF    到期后发送 SIGPROF。
// 当前实现覆盖有界 LTP 所需的读写、非法 which 和坏指针路径。
```

代码 6-2 interval timer 的任务内状态

时间接口目前不是高精度调度器，也没有完整 CPU 时间记账。它的目标是覆盖 `libc` 和 bounded LTP 的可观察语义：能读当前时间，能 `sleep`，能设置 interval timer，能在到期时向任务投递对应信号，能对非法参数返回正确 `errno`。后续若扩展更复杂的定时器测试，需要补齐 `virtual/prof` CPU 时间统计和更精细的 timer slack。

6.4 兼容性状态表

类别	状态	说明
进程生命周期	partial	<code>clone/execve/exit/wait4/yield</code> 有真实状态支撑，作业控制和 <code>ptrace</code> 类能力缺失。
内存接口	partial	<code>brk</code> 、匿名 <code>mmap</code> 、部分 <code>mprotect/munmap</code> 、 <code>SysV shm</code> 可用，完整 <code>page cache</code> 未完成。
文件路径	partial	<code>openat/stat/readlink/symlink/access</code> 等覆盖当前 LTP，完整 <code>mount</code> 和权限模型未完成。
FD I/O	partial	普通文件、 <code>pipe</code> 、伪设备、部分 <code>socket</code> 可用，高吞吐和共享 <code>offset</code> 仍需优化。

类别	状态	说明
信号	<code>partial</code>	基本 <code>action</code> 、 <code>mask</code> 、 <code>pending</code> 、 <code>sigtimedwait</code> 、 <code>sigreturn</code> 可用，实时队列和完整 <code>ucontext</code> 缺失。
网络	<code>stub-risk</code>	本地 <code>socket</code> 队列可用，没有真实外部 TCP/IP 数据路径。

6.5 errno 策略

在真实评测里，返回值常常比“有没有实现”更重要。LTP 很多 case 只触发失败路径，例如坏指针应返回 `EFAULT`，无权限应返回 `EACCES` 或 `EPERM`，符号链接循环应返回 `ELoop`，不存在路径应返回 `ENOENT`，目录类型错误应返回 `ENOTDIR` 或 `EISDIR`。如果内核为了让程序继续运行而无条件返回 0，短期可能绕过一个脚本，长期会让定位变得困难。

因此 wll0s 的兼容策略更倾向于：已覆盖路径返回真实结果；明确不支持的 syscall 返回 `ENOSYS`；参数非法返回可解释 `errno`；只在 `libc` 启动或评测必须接受的地方提供有限 `compatibility success`。这个策略不完美，但比大量 `silent success` 更适合后续维护。

场景	推荐返回	说明
未分发 syscall	<code>ENOSYS</code>	让调用者和日志明确知道能力缺失。
用户坏指针	<code>EFAULT</code>	不能当作空字符串、空结构或成功处理。
flag 组合非法	<code>EINVAL</code>	如 <code>clone flag</code> 依赖关系、 <code>mmap prot/flags</code> 冲突。
权限不足	<code>EACCES/EPERM</code>	路径访问与身份切换需要区分语义。
资源不足	<code>ENOMEM/EMFILE</code>	内存和 <code>fd</code> 上限应返回明确错误。

6.6 跨架构兼容细节

RISC-V64 和 LoongArch64 在大多数 Linux syscall ABI 上相似，但仍存在参数顺序、寄存器、TLS 设置、入口准备和平台定时器差异。`sys_clone` 中 LoongArch64 对 TLS 和 `child tid` 的参数位置就需要特殊处理；`trap frame` 的准备也通过架构条件分支收敛到 `prepare_user_trapframe`。如果这些差异散落在各个模块中，后续双架构回归会很难维护。

```
#[cfg(target_arch = "loongarch64")]
let (tls, child_tid) = (arg4, arg3);

#[cfg(not(target_arch = "loongarch64"))]
let (tls, child_tid) = (arg3, arg4);

if (clone_bits & CLONE_SETTLS) != 0 {
    child_tf[TrapFrameArgs::TLS] = tls;
}
```

代码 6-3 clone 参数的架构差异处理

文档中保留这一点,是因为双架构并不是简单“换 target 编译”。同一个 syscall 在不同架构上可能有 ABI 细节差异,同一个用户程序也可能因为动态链接器或平台计时器差异暴露不同问题。后续每个新 syscall 或新测试组,都应至少考虑 RISC-V64 与 LoongArch64 的寄存器约定和运行日志差异。

第 7 章 LTP 与评测模块

7.1 harness 设计

harness 是 `feat/ltt` 分支区别于普通教学内核的关键模块。它不只是启动一个 `/init`，而是扫描镜像中的 `glibc/musl` 根目录，按测试组运行真实脚本或二进制，并在串口中输出 `judge` 能识别的分组边界。对于 bounded LTP，harness 还支持通过编译期环境变量 `LTP_CASES` 启动一个聚焦 collector，只运行指定 case。没有 `LTP_CASES` 时，仍可回落到真实 LTP 脚本路径。

```
fn ltp_cases_filter() -> Option<&'static str> {
    match option_env!("LTP_CASES") {
        Some(filter) => {
            let filter = filter.trim();
            if filter.is_empty() || filter.eq_ignore_ascii_case("all") {
                None
            } else {
                Some(filter)
            }
        }
        None => None,
    }
}

fn run_ltp_collection_harness(script_path: &str, cases_filter: &str) -> bool {
    let Some((root, _logical_script)) = logical_path_for_script(script_path) else {
        return false;
    };
    let group_name = ltp_group_name(&root);
    console_write("#### OS COMP TEST GROUP START ");
    console_write(group_name);
    console_write(" ####\n");

    for part in cases_filter.split(',') {
        let case = part.trim();
        if !case.is_empty() {
            run_ltp_case(&root, case);
        }
    }

    console_write("#### OS COMP TEST GROUP END ");
    console_write(group_name);
    console_write(" ####\n");
    true
}
```

代码 7-1 LTP case collector

这段代码显示出一个重要原则：bounded LTP 不是在内核里伪造 TPASS，而是仍然调用 `/ltp/testcases/bin/<case>` 的真实二进制。collector 只是负责选择 case 和包装分组输出。每个 case 的通过或失败仍由程序自己输出，最后由 judge 脚本解析。这样可以减少“为了过 parser 而跳过真实语义”的风险。

7.2 测试集合

当前分支的测试并非只有 LTP。基础测试验证 fork、clone、wait、pipe、brk、mmap 等最小能力；BusyBox 验证 shell、路径、文件和脚本执行；Lua 验证解释器和 libc 运行时；iozone 验证文件 I/O、共享内存和多进程吞吐；libc-test 验证 libc ABI 广度；libcbench 和 lmbench 验证性能压力；bounded LTP 则验证更细的 Linux errno 和边界语义。

测试组	目标	主要压力点
basic	基础功能	进程、管道、等待、堆、匿名映射、基本文件。
busybox	用户态工具链	shell、脚本、路径解析、目录项、poll/select。
lua	解释器运行	动态链接、文件 I/O、时间、随机数、futex。
iozone	文件吞吐	ext4 写缓存、pread/pwrite、pwritev/preadv、SysV shm。
libc-test	libc 兼容	信号、线程、内存映射、errno、动态链接。
lmbench	系统性能	fork/exec、pipe、上下文切换、文件和时间接口。
LTP	边界语义	身份、权限、符号链接、readlink/lstat、interval timer。

7.3 有界 LTP 选择

当前 FOCUSED_LTP_CASES 的选择集中在四类能力。第一类是向量写和 fd 路径，例如 writev01；第二类是身份与组，例如 setegid02、getgroups01、setgroups01 到 setgroups04；第三类是文件权限与符号链接，例如 access01、faccessat01、open02/open03、symlink*、readlink*、lstat*；第四类是 interval timer，例如 setitimer01/02 和 getitimer01/02。这些 case 有共同特点：它们能明确验证一个子系统的真实语义，同时不会把内核拖入完整 LTP 的庞大依赖面。

后续扩展 LTP 时，应按“一个模块，一个边界，一组 case”的方式推进。例如扩展文件权限时，应先补齐 VFS 权限和 errno，再加入更多 open/access 组合；扩展时间时，应先补 virtual/prof CPU 时间，再加入更复杂的 timer case；扩展信号时，应先完善 pending queue 和 ucontext，再加入实时信号或 sigaltstack 类测试。这样文档、代码和测试证据能保持同步。

7.4 parser 分组协议

评测脚本依赖串口日志中的分组边界。harness 输出 OS COMP TEST GROUP START 和 OS COMP TEST GROUP END，judge 脚本再根据组名选择对应 parser。这个协议看似只是日

志格式，实际上决定了测试结果是否能被评测系统识别。如果组名错误、END 缺失、用户程序输出被内核日志打断，都会导致 parser 误判。

```
#### OS COMP TEST GROUP START ltp-glibc ####
RUN LTP CASE access01
...
#### OS COMP TEST GROUP END ltp-glibc ####

#### OS COMP TEST GROUP START ltp-musl ####
RUN LTP CASE access01
...
#### OS COMP TEST GROUP END ltp-musl ####
```

代码 7-2 串口分组输出格式

因此内核日志要尽量区分调试信息和用户输出。开发时可以打开更多 `syscall log`，但提交或验收时应保证 `judge parser` 看到的 `START`、`END`、`case` 输出和结果行稳定存在。对 LTP 来说尤其要避免内核主动打印伪造的 `TPASS` 或 `TFAIL`，因为这会破坏“真实用户程序输出”的证据链。

7.5 新增 case 准入标准

新增 LTP case 前，应先回答三个问题：它验证哪个内核对象；失败时最小复现是什么；通过后会破坏已有测试组。一个 case 可能表面属于文件系统，但实际依赖用户指针、凭据、路径解析、VFS 后端和 `signal` 中断。把依赖拆清楚后再实现，能减少“修一个 case 坏三个 group”的情况。

阶段	动作	验收点
选择 case	只选一个相邻子系统	能说明目标 <code>syscall</code> 、 <code>errno</code> 和依赖对象。
最小运行	<code>--suite ltp</code> 加 单个 <code>LTP_CASES</code>	串口日志中能看到真实二进制输出。
实现修复	补真实语义而不是伪成功	失败路径和成功路径都有明确 <code>errno</code> 。
回归验证	跑相关 <code>basic/busybox/lua/benchmark</code>	新增 case 不破坏已有默认路径。
纳入集合	更新 <code>FOCUSED_LTP_CASES</code> 和文档	case 名、证据和边界描述保持同步。

第 8 章 测试结果与问题分析

8.1 编译与运行方式

Typst 文档生成与内核测试是两条不同路径。本文档使用 `typst compile` 生成 PDF；内核本身使用 `make` 和 QEMU 运行。`feat/ltip` 分支提供了多种脚本，既可以跑默认 `all profile`，也可以只跑 `focused LTP`。为了便于复现，文档列出最小命令，不要求读者一次性运行所有慢速评测。

```
# 生成本文档 PDF
typst compile typst-doc/wllos-ltp.typ typst-doc/build/wllos-ltp.pdf

# 打印当前 focused LTP case 列表
make -C ltp print-ltp-cases

# 构建 RISC-V64 内核并打开 LTP feature
make -C ltp ARCH=riscv64 build LTP=1

# 运行 focused LTP，保留串口日志供 judge 脚本解析
python ltp/scripts/perf_baseline_runner.py --arch riscv64 --suite ltp --runs 1
```

代码 8-1 文档与内核的常用命令

由于评测运行依赖镜像、QEMU、工具链和运行时间，本文档不把测试输出硬编码成“永远成立”的结论。更推荐的做法是在每次大改后保存串口日志、runner summary 和 judge 解析结果，并在 README 或开发日志中记录 `commit`、架构、suite、命令和结果。这样一旦出现回归，能够快速定位是内核语义、构建配置、镜像内容还是 parser 变化。

8.2 当前能力总结

结合仓库中的 `syscall matrix` 和评测脚本，当前分支已经具备以下可观察能力：基础进程生命周期可运行，匿名内存和 heap 可支撑 `libc`，`ext4/MemFS/fd/pipe` 能支撑 `BusyBox` 和文件测试，动态 ELF 与解释器路径能启动 `glibc/musl` 用户程序，`bounded LTP case` 有真实二进制路径，`libc-test`、`iozone`、`libcbench`、`lmbench` 都有对应 harness 入口和 judge 脚本。

能力	已具备内容	主要风险
任务	<code>clone/execve/wait4/exit_group</code>	完整 <code>pthread</code> 、作业控制、 <code>ptrace</code> 和进程组等待仍缺失。
内存	<code>brk/mmap/munmap/mprotect/shm</code>	<code>dirty tracking</code> 、完整 <code>page cache</code> 和精确 <code>accounting</code> 仍不完整。
文件	<code>open/read/write/stat/symlink/readlink</code>	跨后端 <code>rename/link</code> 、完整权限和持久化语义仍有限。

能力	已具备内容	主要风险
信号	sigaction/ sigprocmask/ sigtimedwait	实时信号队列、完整 ucontext 和 syscall restart 未完成。
时间	clock_gettime/ nanosleep/setitimer	virtual/prof timer 的 CPU 时间统计仍粗略。
网络	本地 socket 队列	无真实外部网络栈，不适合声称 iperf/netperf 完整支持。

8.3 典型问题定位

在这个分支中，失败通常不会只属于单个 syscall。一个 LTP access 失败可能来自路径解析、父目录搜索权限、fsuid/fsgid、VFS metadata 或用户指针复制；一个 BusyBox 脚本失败可能来自 shebang、execve 参数栈、动态解释器、/proc/self/exe 或 fd close-on-exec；一个 iotest 失败可能来自 ext4 写缓存、SysV shm、pwritev、fork 后 fd offset 或 wait4。定位时应先看串口日志中的最后一个用户程序，再看 syscall debug，再用 focused case 缩小范围。

现象	优先检查	原因
程序启动即失败	execve、ELF loader、用户栈、解释器	大部分 libc 问题会在入口前后暴露。
路径 errno 不对	VFS 解析、父目录权限、符号链接	LTP 对 ENOENT/EACCES/ELOOP 很敏感。
等待卡住	WaitQueue、timer、signal、child exit	可能是丢失唤醒或 stale waiter 没清理。
文件吞吐异常	fd offset、write cache、pwritev、sync	iotest/lmbench 更容易放大性能与语义问题。
LTP 单 case 失败	先用 LTP_CASES=case 单独运行	避免多个 case 交织导致日志难读。

8.4 回归证据记录格式

为了让测试结果可复现，建议每次关键运行都记录同一组字段。只写“通过”没有太大价值，因为无法区分架构、镜像、命令、case 集和 parser 是否一致。比较可靠的记录方式应包含 commit、架构、构建 feature、运行命令、QEMU 退出码、串口日志路径、parser summary 和失败 case 列表。

字段	示例说明
提交	feat/ltp 分支具体 commit，例如本地 checkout 的短 hash。
架构	riscv64 或 loongarch64，不要混写。
构建选项	LTP=1、LTP_CASES=...、IOZONE=1、LMBENCH=1 等。
运行命令	完整 runner 命令，包含 suite、runs 和是否删除 sdcard copy。

字段	示例说明
日志路径	串口日志、runner summary、judge 输出的保存位置。
结论	通过数量、失败数量、失败 case、首个异常 syscall 或 panic。

```
commit: 583680e
arch: riscv64
suite: ltp
case: access01
command: python scripts/perf_baseline_runner.py --arch riscv64 --suite ltp --runs
1
kernel: LTP=1 LTP_CASES=access01
evidence:
- qemu exit code
- serial log path
- judge_ltp summary
- first failing syscall if any
```

代码 8-2 推荐的单 case 调试记录

这类记录可以直接贴进开发日志，也可以后续转为自动生成表格。它的价值在于把“测试通过”变成可检查证据：当未来某个模块改动导致回归时，开发者可以用同一命令、同一 case 和同一 parser 复跑，而不需要重新猜测当初的环境。

第 9 章 总结与展望

9.1 总结

wll0s feat/ltp 分支已经从普通教学内核推进到面向真实评测的内核形态。它的主要特点不是一次性覆盖 Linux 全部接口，而是围绕真实用户程序建立了一组可运行、可解释、可回归的核心语义：地址空间以 MemorySet 和 VMA 为中心，任务以 TaskControlBlock 和 thread group 为中心，文件系统以 fd/VFS/backing object 为中心，系统调用以 Linux id 分发和 errno 为中心，harness 以真实测试程序输出为中心。

与 UndefinedOS 文档相比，本文档保留了正式技术文档的排版结构：封面、自动目录、页眉页脚、章标题、节标题、表格、图注、灰底代码块和代码编号；内容上则更贴近 wll0s 的当前分支，把旧稿中分散的章节合并进模块内部。这样章节数量更少，阅读路径更清楚，但每个模块仍然有足够的设计说明和关键代码，后续可以继续沿着模块扩写。

9.2 后续工作

后续工作建议按优先级推进。第一，继续强化 VFS 和 fd 语义，尤其是共享 open-file description、跨后端 rename/link、权限检查和持久化边界。第二，完善线程和 futex，补齐 pthread 常见路径、robust futex、共享 futex key 和取消语义。第三，改进 signal 和 timer，包括完整 ucontext、实时信号队列、syscall restart 和 virtual/prof CPU 时间。第四，在文件 I/O 上补 dirty tracking、msync、稀疏文件和更高效的 pwritev/preadv。第五，在网络方面，不应急于声称 iperf/netperf 支持，而应先建立真实 virtio-net 或最小 TCP/IP 数据路径。

LTP 扩展仍应保持克制。每增加一组 case，都应明确它验证的内核对象、需要的 errno、涉及的用户指针和可能影响的已有测试。只有当真实二进制、串口日志、judge parser 和回归 suite 都能给出稳定证据时，才应把该 case 纳入默认集合。这样的节奏虽然慢，但能避免内核被兼容分支和伪成功输出拖乱，也更符合一个可维护竞赛内核的长期目标。